

Telegram Stock Query Assistant

COMP7940 Project Report

Name: Yishi Qiu
Student ID: 25421018

1. Introduction

Chatbot ID: 889426008

Telegram Bot Username: @comp7940_YishiQiu_bot

Telegram Bot Link: https://t.me/comp7940_YishiQiu_bot

In this project, a cloud-based Telegram chatbot called Telegram Stock Query assistant is developed. The chatbot will help answer natural-language stock questions and manage a personal watchlist with Telegram. The project is based on the COMP7940 Cloud Computing project specifications where cloud deployment, database logging, integration of the LLM, containerization, Git-based development, and observability are stressed instead of merely the chatbot capability.

The project has two main features mentioned in the public repository: natural-language stock query and personal watchlist management. It also indicates that the project was developed using Python 3.12, FastAPI, SQLAlchemy, MySQL, Telegram Bot API, MiMo OpenAI-compatible API, Twelve Data stanagagger and a docker-compose service with git actions support for a Prometheus-compatible metrics endpoint.

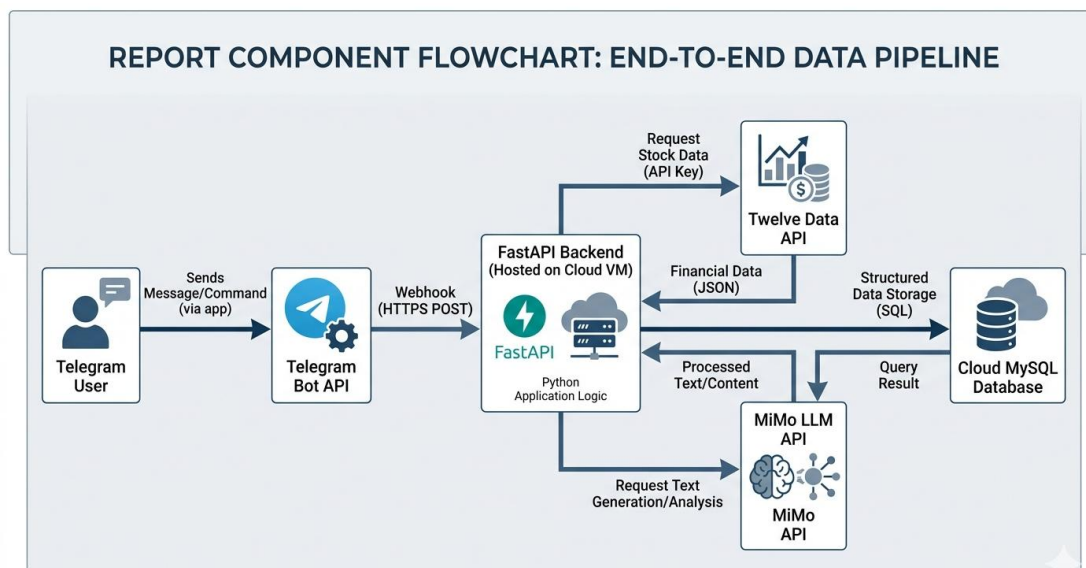
2. Application Summary

The primary objective of this project is to offer a lightweight yet a complete cloud chatbot service. Rather than providing any fixed or programmable responses, the chatbot takes Telegram users natural-language queries, accesses stock-related data via an external data API, and generates brief summaries via an LLM API. Moreover, the app enables individual users to have their own watchlist, which makes the service more useful and tenacious than a mere query bot.

According to the repository README, the first feature supports queries such as “AAPL today”, “Compare AAPL and MSFT”, and “How is Nvidia stock doing?”. The second feature supports watchlist operations including /add, /remove, /watchlist, and /summary.

3. System Design

It is a Telegram-based cloud service. When a user sends the Telegram bot a message, the backend takes the request and identifies the user intent and retrieves stock information where necessary, calls the LLM API to summarize the text, logs the messages to the database, and sends the result to Telegram. This design is a direct reflection of the project brief according to which the chatbot is supposed to accept natural-language input and use an LLM and track requests and responses in order to debug and analyze them.



The project repo indicates a FastAPI back-end with a Dockerized deployment architecture. The Dockerfile will boot up the application on port 8000 with uvicorn app. main:app. Docker-compose. yml, monitoring configuration, Kubernetes-related files, and a GitHub Actions workflow are also present in the repository, which, in turn, evidences the readiness to deploy and the design of the workflow based on DevOps.

4. Main Features

4.1 Natural-Language Stock Query

The former is the first key feature that enables users to query stock information using Telegram in plain English. The chatbot can answer single-stock questions and two-stock comparisons in a single request. It calls the stock data API to get market information, and then calls the LLM API to produce a succinct explanation to the user. It enhances usability since users do not have to manually interpret raw numbers or structured API responses.

4.2 Personal Watchlist Management

The second key characteristic enables users to have their own watchlist. The users are able to add and remove stocks, view the existing watchlist, and order a summary of followed stocks. This is a characteristic that adds enduring user data and illustrates how the database will be practically utilized in the application. It also renders the project more precise as a real service instead of a one time query tool.

5. Technical Requirements Fulfillment

This section discusses how the project meets the mandatory technical requirements as contained in the course brief.

First, the chatbot is deployed on Telegram, which meets the requirement that the bot should be on Telegram. The bot profile and Telegram chat screenshots can be used as evidence.

Second, the application stores persistent data and request logs in a cloud database, which meets the criterion that the app should have a database system with a cloud provider to log data. The repository says that the deployed version is a cloud version of MySQL database.

Third, the app is cloud-hosting ready. The README clearly mentions that the version deployed in the end is on a cloud VM and cloud MySQL database and that this meets the requirement that the application should be deployed on a cloud platform.

Fourth, the project generates responses with an LLM API. The repository captures the MiMo OpenAI-compatible API in the technology stack and environment setup, meeting the criteria that the app needs to consume an LLM API.

Fifth, Git manages the project with a publicly accessible GitHub repository. The repository contains versioned source code, configuration files, tests, and workflow files, which satisfies the Git management need.

Sixth, the project leverages container technology. This repo includes a dockerfile and docker-compose. yml. The Dockerfile packages FastAPI backend as a container, while Docker Compose defines multiple services for executing the application and supporting observability. This satisfies the containerization requirement.

At last, the project also covers monitoring and cost aware cloud management. The repository contains a monitoring directory, which spin up some services that interact with Prometheus/Grafana, as well as a endpoint for metrics compatible with Prometheus

in the documented stack. This substantiates that the project requirement of observability and management at low cost could be met by doing so.

6. DevOps and Nice-to-Have Considerations

Apart from the essentials, the course specification awards extra points for making a real DevOps workflow complete, scalability, load balancing, orchestration as well as security and other practical considerations.

This project tackles these elements in different ways. The proof is in the pudding: a GitHub Actions workflow file shows that CI is also supported. Also some Kubernetes-related files as well as a mention that the backend is stateless and amenable to horizontal scaling. Deployment / Operations section of the README emphasizes load balancing, orchestration, environment-based secret management, health checks, metrics and fallback stock data sources.

While this project is merely an educational prototype and not a production system by any means, these design elements show some awareness for cloud-native architecture and some practical DevOps thought.

7. Job Division

This project was completed individually.

The completed tasks include project design, feature definition, Telegram bot implementation, backend development, LLM API integration, database integration, Docker-based containerization, deployment preparation, monitoring setup, testing, documentation, and presentation preparation.

8. Cloud Resource Charges

The course brief requires the report to include the total cloud resource charge.

The resources used in this project include:

1. a cloud VM for hosting the backend service,
2. a cloud database instance,
3. LLM API usage,
4. related network or monitoring resources if applicable.

Cloud VM: 34 RMB/month

Cloud MySQL Database: 34 RMB/month

Estimated total monthly cloud cost: about 68 – 70 RMB/month
LLM API cost: 35 per month
Total estimated cost for this project deployment: 5RMB

Xiaomi MIMO

联系我们 文档 Token Plan 控制台 博客 中文

账户余额
API Keys
账单明细
充值明细
发票开具
插件管理
订阅管理

套餐使用情况

Lite

有效期至 2026-05-13 23:59 (UTC)

当前套餐用量

1,008,172 / 60,000,000 Credits

已使用 1.7%

开具发票

订阅套餐

订单信息

子订单号	产品	资源类型	规格	单价	数量	付...	实付金额
20260415541139575643141	轻量数据库服务LighthouseDB 资源ID: lhdbmysql-ins-pexmj4oz	包年包月	计费模式: 包年包月 配置: 1核1GB内存20G存储, MySQL8.0 地域: 香港 可用区: 香港二区	35.00 元/月	x1	按...	35.00

订单实付: 35.00元

订单信息

子订单号	产品	资源类型	规格	单价	数量	付...	实付金额
20260414541139562470571	轻量应用服务器 资源ID: lhins-rndux1j5	包年包月	运算组件: 2核CPU、2GB内存 (中国香港入门型Linux-2核2G-40G-0.5T) 系统盘: 40GB SSD云硬盘 (中国香港入门型Linux-2核2G-40G-0.5T) 流量包: 0.5TB 月流量包 (中国香港入门型Linux-2核2G-40G-0.5T) 地域: 中国香港 镜像: Ubuntu 24.04 LTS	38.00 元/月	x1	按...	38.00

订单实付: 38.00元

The deployment was intentionally designed to remain lightweight and low-cost, so that the system could satisfy the required cloud functionality while keeping expenses minimal.

9. Evidence List

To support the technical requirements, the following evidence should be attached in the report:

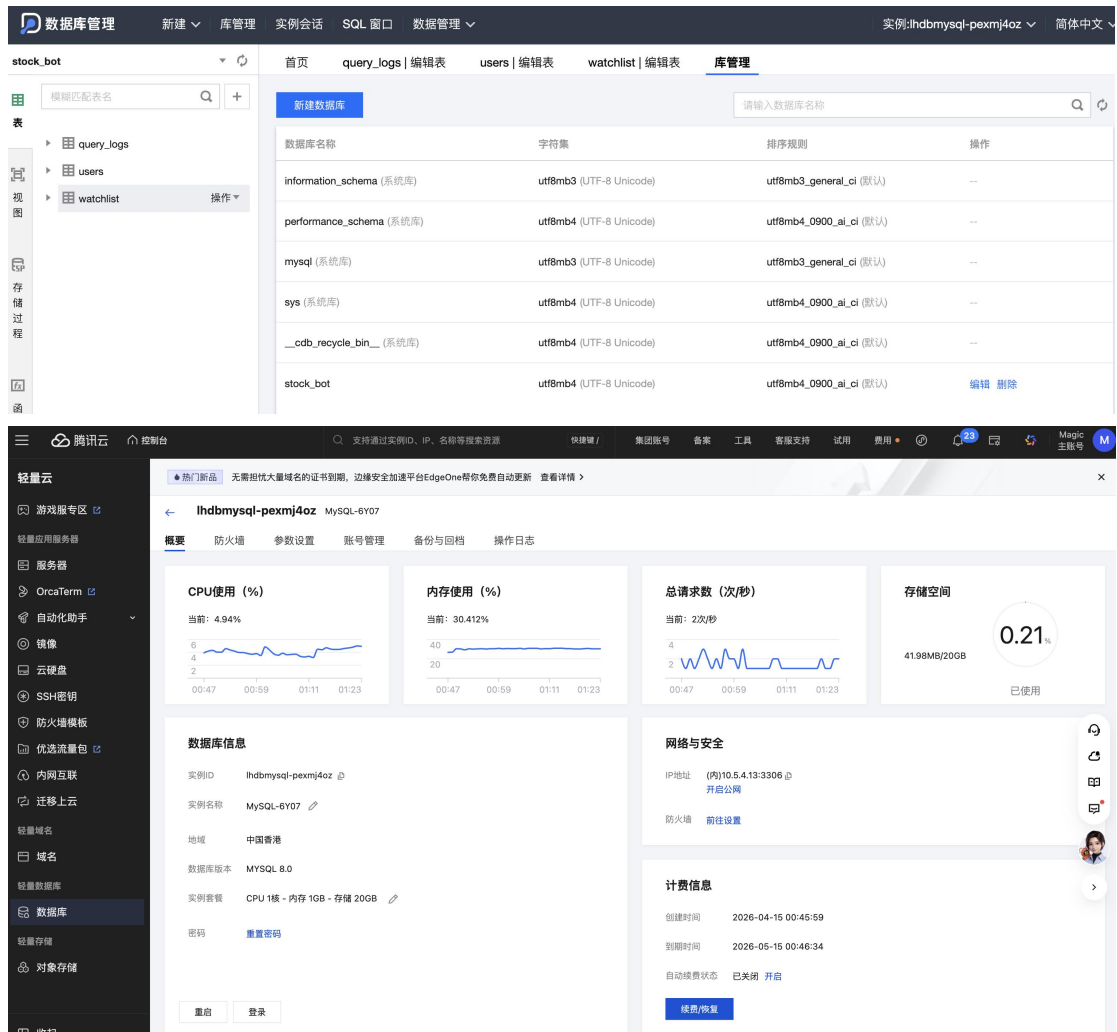


Figure 1. Cloud MySQL database instance and stock_bot schema.



Figure 2. Cloud VM instance running on Tencent Cloud.



Figure 3. Telegram bot startup message.



Figure 4. Telegram stock query result for AAPL.

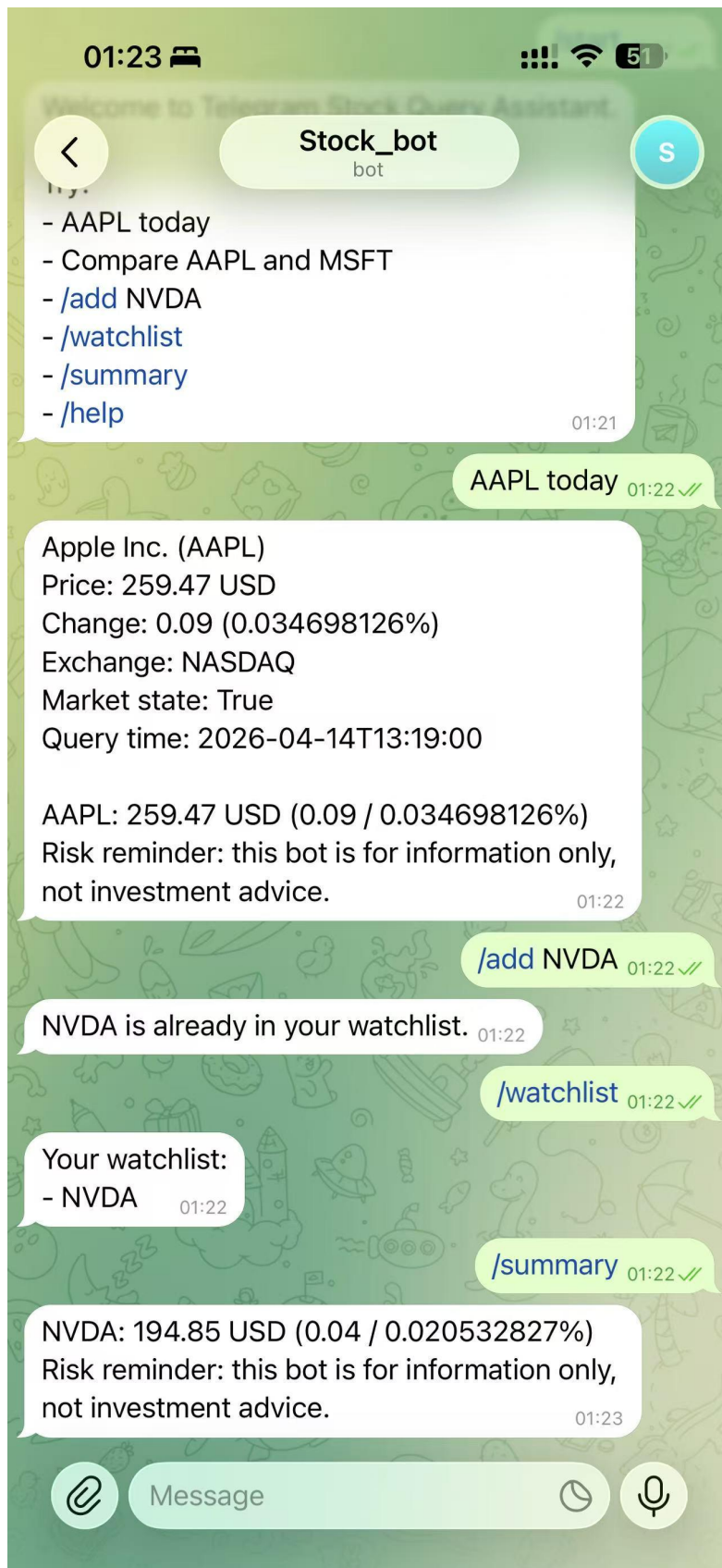


Figure 5. Telegram watchlist interaction.

数据库管理

新建

库管理

实例会话

SQL 窗口

数据管理

stock_bot

模糊匹配表名

+

表

query_logs

列

id

user_id

intent

user_message

detected_symbols

market_data

llm_reply

status

error_message

created_at

索引

users

watchlist

视图

存储过程

函数

触发器

事件

query_logs | 编辑表

新增

删除

导出

复制

提交

快捷操作 >>

id	user_id	intent	user_message
1	1	start	/start
2	1	start	/start
3	1	start	/start
4	1	stock_query	AAPL today
5	1	add_watchlist	/add NVDA
6	1	show_watchlist	/watchlist
7	1	summary_watchlist	/summary
8	1	help	/help
9	1	show_watchlist	把中国银行的股...
10	1	unknown	移除NVDA
11	1	start	/start
12	1	stock_query	AAPL today
13	1	add_watchlist	/add NVDA
14	1	show_watchlist	/watchlist
15	1	summary_watchlist	/summary

数据库管理

新建

库管理

实例会话

SQL 窗口

数据管理

stock_bot

模糊匹配表名

+

表

query_logs

列

id

user_id

intent

user_message

detected_symbols

market_data

llm_reply

status

error_message

created_at

索引

users

watchlist

视图

存储过程

函数

触发器

事件

users | 编辑表

新增

删除

导出

复制

提交

快捷操作 >>

id	telegram_user_id	username	first_name
1	8224873227	(NULL)	Ling

显示总数

50 条 / 页

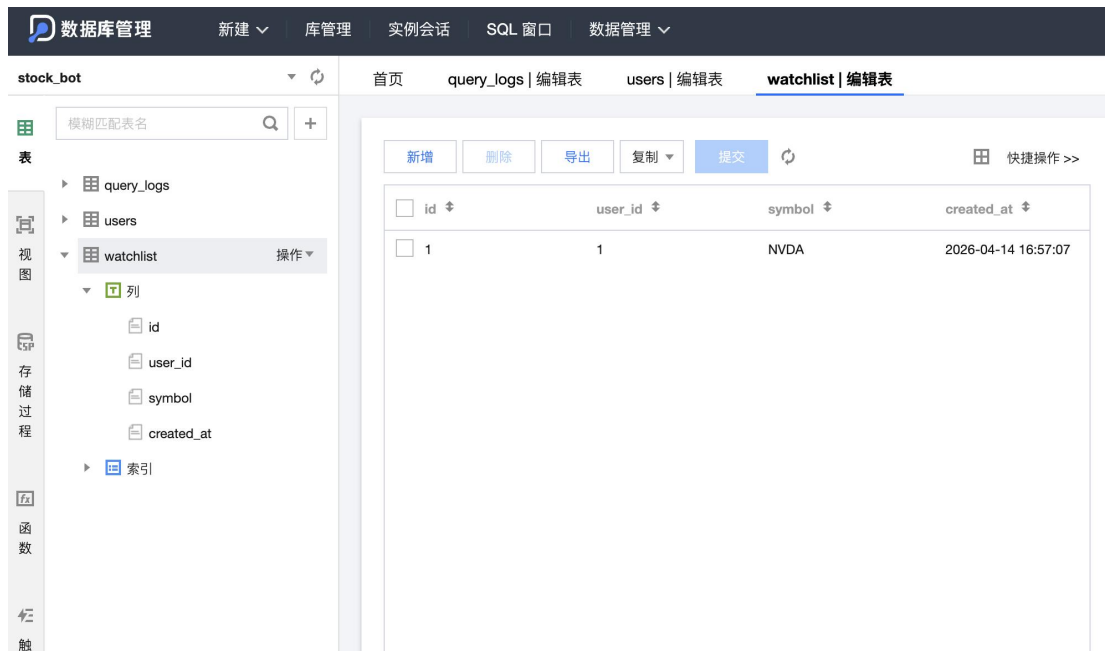
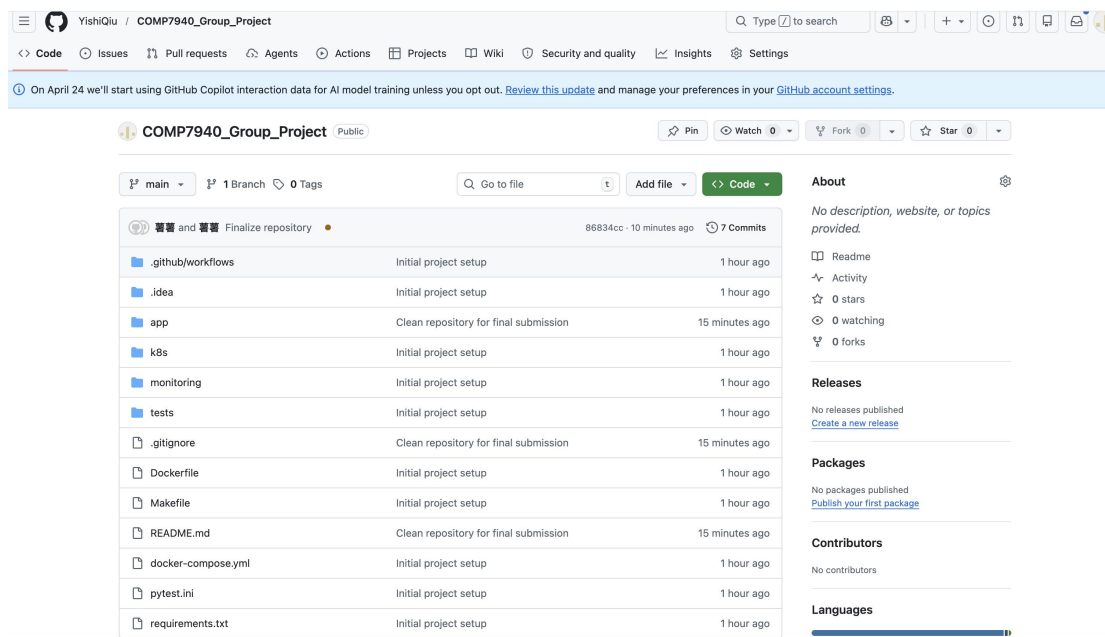


Figure 6. Query log table and watchlist table in the cloud database.



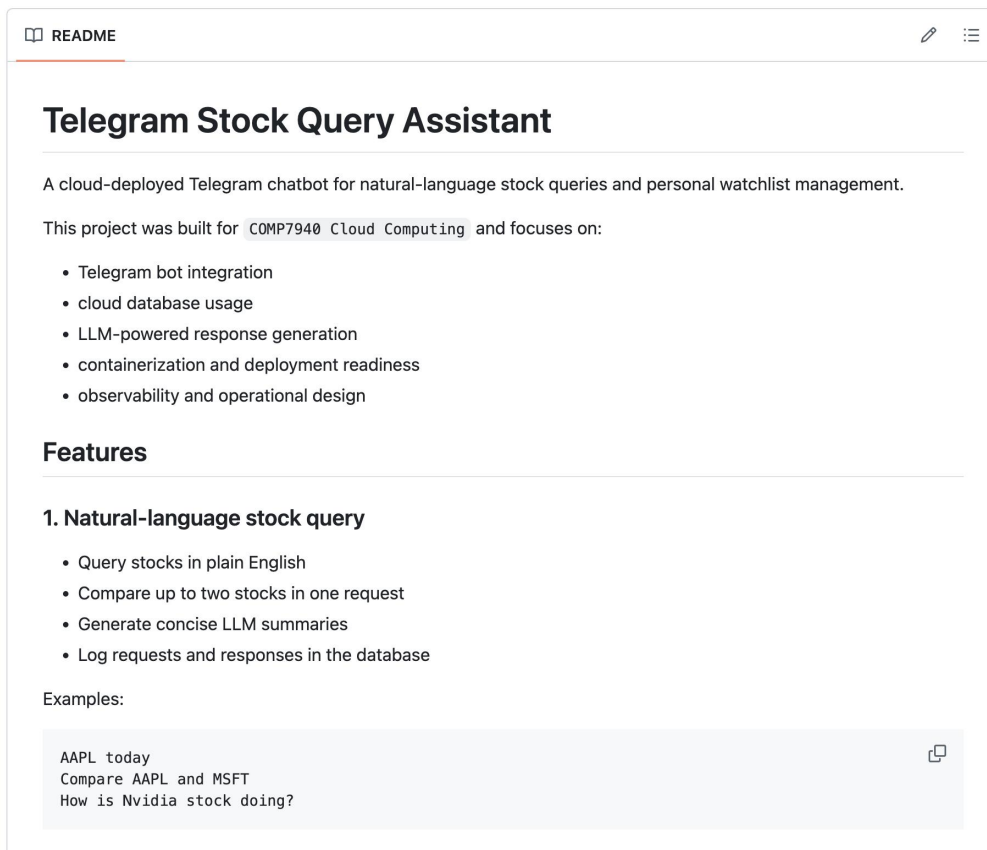


Figure 7. GitHub repository homepage.

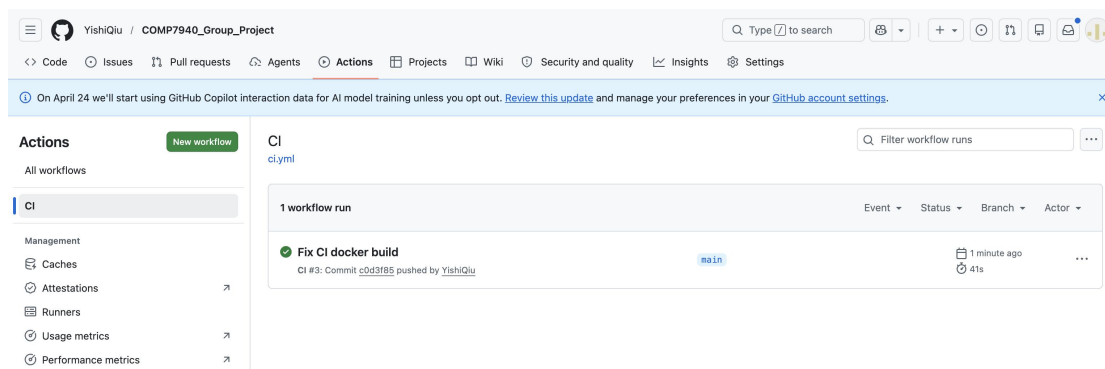
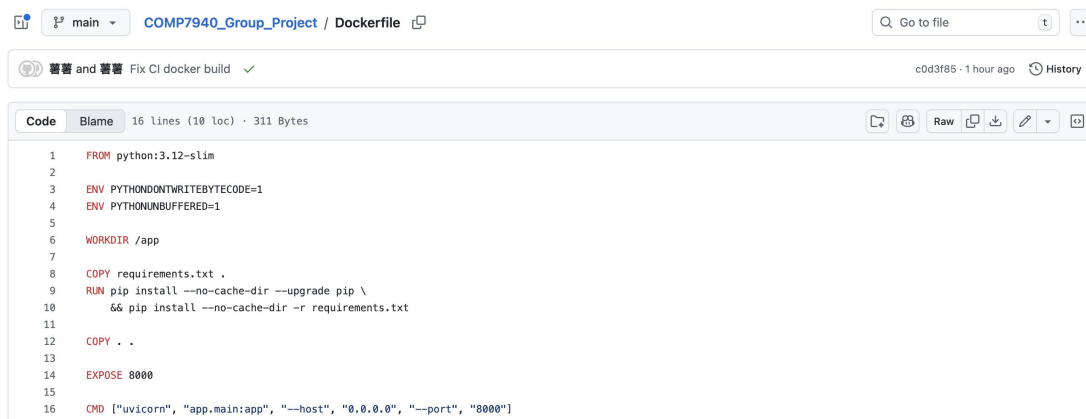


Figure 8. GitHub Actions CI workflow.



```
1 FROM python:3.12-slim
2
3 ENV PYTHONDONTWRITEBYTECODE=1
4 ENV PYTHONUNBUFFERED=1
5
6 WORKDIR /app
7
8 COPY requirements.txt .
9 RUN pip install --no-cache-dir --upgrade pip \
10     && pip install --no-cache-dir -r requirements.txt
11
12 COPY . .
13
14 EXPOSE 8000
15
16 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 9. Docker container running the FastAPI backend service.

```
(.env) root@VM-0-5-ubuntu:/home/ubuntu/COMP7940_Group_Project# curl http://127.0.0.1:8000/metrics | head -n 30
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed

10# HELP python_gc_objects_collected_total Objects collected during gc--      0
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 484.0
python_gc_objects_collected_total{generation="1"} 20.0
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 260.0
python_gc_collections_total{generation="1"} 23.0
python_gc_collections_total{generation="2"} 2.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="12",patchlevel="3",version="3.12.3"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 2.78859776e+08
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 9.0710016e+07
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.77618577294e+09

(.env) root@VM-0-5-ubuntu:/home/ubuntu/COMP7940_Group_Project# curl http://127.0.0.1:8000/healthz
curl http://127.0.0.1:8000/readyz
{"status":"healthy"}{"status":"ready"}(.env) root@VM-0-5-ubuntu:/home/ubuntu/COMP7940_Group_Project#
```

Figure 10. Monitoring / metrics evidence for observability.

10. Conclusion

In conclusion, this project presents a practical cloud-based Telegram chatbot for stock queries and personal watchlist management. The project integrates Telegram as the user

interface, a cloud-hosted FastAPI backend service, a cloud MySQL database for logging and persistence, an external stock data API, and an LLM API for summarization. The repository structure further demonstrates containerization, monitoring considerations, and DevOps-oriented deployment readiness.

Overall, the project satisfies the main technical goals of the COMP7940 course project and demonstrates how cloud computing concepts can be applied to a practical chatbot service.